Core Concepts of Agentic AI Systems

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Recap:

1. LangGraph
2. Vanilla RAG
3. Agentic RAG

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

LangGraph Basics - The Three Building Blocks

Great Learning
POWER AHEAD

State

Shared working memory of the graph - a typed dict (**TypedDict**) that flows through every step.

Every node reads from it and writes its results back to it.

Example fields: user_query, retrieved_docs, grade, route and more.

Nodes

Plain Python functions - each does one unit of work, then returns the updated state.

Nodes can call LLMs, tools, vector stores, or pure logic.

Examples: validate_input, retrieve_context, generate_answer...

Edges

Wiring between nodes - they define which step runs next.

Linear edges give a fixed sequence; conditional edges branch at runtime.

Compiled with graph.compile() into a runnable app.

LangGraph = an explicit state machine for LLM workflows: typed state + node functions + edges that decide the route.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Routing - Linear vs Conditional Edges

add_edge() = fixed path. add_conditional_edges() = branch at runtime.
This runtime choice is where workflows become agentic.

➡️ Linear Edges

How it works:

- graph.add_edge("A", "B") - B always follows A
- Deterministic sequence, same route every run

Examples:

- ingest → chunk → embed → index pipelines
- Good for steps that must always happen, in order

🔀 Conditional Edges

How it works:

- A router function inspects the state, returns a label
- The label maps to the next node to run

Examples:

- Validation gates, retrieval-grade routing
- Enables early exits and fallbacks

Conditional edges are where a pipeline gains judgment:
gates, fallbacks, and self-correction all live on these branches.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Vanilla RAG - The Fixed Three-Step Pipeline

1. Indexing

Load documents, split into overlapping chunks, embed each chunk into a vector

Store vectors + metadata in a vector database

Example stack: PDFs → overlapping chunks → embedding model → ChromaDB / FAISS

2. Retrieve

Embed the user query with the same model and run a semantic similarity search

Return the top-k most relevant chunks as context

Example: similarity_search(query, k=4) - cosine similarity over the index

3. Generate

Stuff the retrieved chunks into the prompt and let the LLM answer grounded in them

Grounding reduces hallucination and lets you cite sources

Example: an LLM at temperature 0 fills a fixed answer template from the context

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Where Vanilla RAG Falls Short

It always trusts retrieval

Top-k always returns something - even when the KB has nothing relevant.

The LLM then answers from wrong-company context.

No self-check on quality

The pipeline never asks “are these chunks actually sufficient?”

Garbage context flows straight into generation.

Knowledge bases go stale

Curated internal docs are rich but frozen in time.

New data and events simply are not in the corpus.

One fixed path for every query

Retrieve-then-generate runs identically whether the KB covers the query perfectly or has never heard of it.

Vanilla RAG has no decision point - it cannot notice a bad retrieval, and it cannot change course.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Agentic RAG - Retrieve, Grade, Route

Agentic RAG adds a decision point after retrieval: grade what came back, then route to the best data source at runtime.

Retrieve

Same first step as vanilla RAG

- Semantic search over the KB for every entity or sub-question in scope
- Collect top-k chunks per query
- All chunks go to the grader, not straight to the LLM

Grade

Is this retrieval good enough?

- Simple option: a deterministic check - is the target entity in the chunks?
- Verdict: SUFFICIENT or INSUFFICIENT
- Alternative: LLM-as-grader (semantic, but non-deterministic)

Route

The agentic decision

- SUFFICIENT → answer from the internal KB
- INSUFFICIENT → fall back to live web search

The route is chosen at runtime based on the input query

Note: *This is one-of-many Agentic RAG patterns.*
Retrieval can also be exposed as a tool, where the agent itself decides whether and when to retrieve.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Three Scenarios - Three Paths

1. KB Hit

Input: A query fully covered by the internal corpus

Result: Grader finds the target in the retrieved chunks

Grade: SUFFICIENT

Route: internal_kb - no web search

2. KB Miss

Input: A valid query the corpus does not cover

Result: Chunks come back about the wrong topics

Grade: INSUFFICIENT

Route: web_fallback (live web search)

3. Invalid Input

Input: A nonsense or fake entity

Result: Validation gate sets is_valid = False

Grade: Never reached

Route: early exit → error report

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Q&A

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Great Learning

Power Ahead !

